Web Engineering & Development (SWE 363)

RESTful Web APIs

In today's lecture:

- RESTful web APIs with Express
- Connecting the API to MongoDB
- Using the API with Fetch

Reference:

- Zybook: 7.3, 7.4 (parts), 7.5

Activity: Think-Pair-Share (2 min)

1. Which Mongoose method would you use to add a student, list all students, find one student by ID, change a GPA, and remove a student?
   Choose from: `find()`, `findById()`, `create()`, `updateOne()`, `deleteOne()`
2. In 5.17, how did a React app get forecast data from a web API?
3. Which Express method sends a JavaScript object as JSON?

7.3 RESTful Web APIs (Node)

What is a RESTful Web API?

> A **RESTful web API** implements **CRUD** operations on **resources** (data at a URL) using HTTP **verbs**; data is sent as **JSON**

Why: one standard interface for browsers, mobile apps, and other clients

```
POST   /api/students       → Create a new student
GET    /api/students       → Retrieve all students
GET    /api/students/42    → Retrieve student 42
PUT    /api/students/42    → Update student 42
DELETE /api/students/42    → Delete student 42
```

Status Codes

> Every API response includes a **status code** that tells the client what happened

Why: the front end checks it: `response.ok` is true for a **2xx** code

Code	Meaning	Example
200 OK	Success, with data	`GET /api/students`
201 Created	New resource created	`POST /api/students`
204 No Content	Success, **no** body	`PUT , DELETE`
404 Not Found	Resource doesn't exist	`GET /api/students/999`
400	Error (e.g., invalid ID)	any route

Activity: Classify (verb, URL, and status code)

A Movie API has the endpoint `/api/movies`.

Verbs: GET, POST, PUT, DELETE

Status codes: 200 OK, 201 Created, 204 No Content, 404 Not Found

1. List all movies
2. Add a new movie
3. Change the title of movie 7
4. Remove movie 7
5. Get movie 500, which does not exist
6. Which of these requests usually send JSON to the server?

Express Router

> `express.Router` creates modular route callbacks for the API **endpoints** (URLs used to access a resource)

Why: all API routes share one prefix, `/api`

```
const router = express.Router();

// GET request returns JSON-encoded student
router.get("/students", function(req, res) {
  res.json({ name: "Sue Black", gpa: 3.1 });
});

// All requests to API begin with /api
app.use("/api", router);     // → GET /api/students
```

Receiving JSON: `express.json()`

> `express.json()` is middleware that converts a JSON request body into a JavaScript object in `req.body`

Why: POST and PUT send their data as JSON (`Content-Type: application/json`)

```
// Middleware that parses HTTP requests with JSON body
app.use(express.json());

router.post("/students", function(req, res) {
  console.log(req.body);   // { name: 'Jack White', gpa: 2.5 }
  res.sendStatus(201);
});
```

Activity: Predict, then what happens if

```
app.use(express.json());
router.get("/students", function(req, res) {
  res.json([{ name: "Sue" }, { name: "Li" }]);
});
router.post("/students", function(req, res) {
  console.log(req.body);
  res.sendStatus(201);
});
app.use("/api", router);
```

1. What does a GET request to `http://localhost:8080/students` return?
2. A student is POSTed, then `/api/students` is requested. How many students?
3. `Content-Type` is removed from the POST request. What does the server output?
4. `app.use(express.json())` is removed. Can the route still read `req.body`?

7.4 RESTful Web APIs: Using a Database

Routes with Mongoose

> API routes use `async` callbacks that `await` a **Mongoose** method (7.1)

Why: the resources now live in MongoDB, not in the code

```
router.get("/students/:id", async function(req, res) {
  try {
    const stu = await Student.findById(req.params.id);
    if (stu === null) {
      res.sendStatus(404);
    } else {
      res.json(stu);
    }
  } catch (ex) { res.status(400).send(ex.message); }
});
```

One Mongoose Method per Route

> Each CRUD route calls **one** Mongoose method on the `Student` model

Why: every route in the lab follows this pattern

Route	Mongoose method	Code
`GET /students`	`find({})`	200
`GET /students/:id`	`findById(req.params.id)`	200
`POST /students`	`create(req.body)`	201
`PUT /students/:id`	`updateOne({ _id: req.params.id }, req.body)`	204
`DELETE /students/:id`	`deleteOne({ _id: req.params.id })`	204

Activity: Predict the status code (in every route, an exception → 400)

Route	Sends
`GET /api/students/:id`	404 if `findById()` returns `null`, else JSON
`PUT /api/students/:id`	404 if `matchedCount` is 0, else 204
`DELETE /api/students/:id`	404 if `deletedCount` is 0, else 204
`POST /api/students`	201 + JSON

1. `GET /api/students/abc` (not a valid ObjectId)
2. `PUT /api/students/<valid ID>` with `{ "gpa": 3.5 }`
3. `DELETE /api/students/<valid ID, not in the database>`
4. `POST /api/students` with a valid student

7.5 Using RESTful Web APIs with Fetch

Getting Data with `fetch()`

> `fetch()` sends an HTTP request from the browser; `response.json()` parses the JSON response

Why: this is how the front end reads the API's data (as in 5.17)

```
addEventListener("DOMContentLoaded", async function() {
  const response = await fetch("/api/students");
  const students = await response.json();
  let html = "";
  for (let stu of students) {
    html += `<li>${stu.name}</li>`;
  }
  document.querySelector("ul").innerHTML = html;
});
```

Sending Data: POST with `fetch()`

> A second argument to `fetch()` sets the request **method**, **headers**, and **body**

Why: without `Content-Type: application/json`, the server won't accept the JSON

```
const stu = { name: "Jack White", gpa: 2.5 };
const response = await fetch("/api/students", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(stu)
});
if (response.ok) {
  const results = await response.json();
  alert("Added student with ID " + results._id);
}
```

- **PUT:** same, with `"PUT"` and the ID in the URL; **DELETE:** only `method`

Activity: Put the steps in order

A. Mongoose's `Student.create()` saves the student in MongoDB

B. The user clicks Add, and `fetch()` sends a POST with the JSON-encoded student

C. The page checks `response.ok` and shows the new student's ID

D. `express.json()` converts the request body into `req.body`

E. The route sends a 201 response with the JSON-encoded student

F. The `/api` router sends the request to the `/students` POST route

Activity: Exit ticket - find the 4 bugs

```
// server.js
const app = express();
const router = express.Router();
router.post("/students", async function(req, res) {
  try {
    const stu = Student.create(req.body);
    res.status(200).json(stu);
  } catch (ex) {
    res.status(400).send(ex.message);
  }
});
app.use("/api", router);

// page.js
const response = await fetch("/api/students", {
  method: "POST",
  body: JSON.stringify(stu)
});
```

Key Takeaways

- A **RESTful web API** does CRUD on resources: **POST, GET, PUT, DELETE**
- Status codes: **200** OK, **201** Created, **204** No Content, **404** Not Found, **400** error
- `app.use("/api", router)` + `express.json()` → API routes that read `req.body`
- Routes `await` Mongoose: `find`, `findById`, `create`, `updateOne`, `deleteOne`
- `fetch(url, { method, headers, body })` + `response.ok` + `response.json()`

30m

Demo

7.2 RESTful APIs

- Songs API: Express + Mongoose CRUD routes
- React front end with Fetch

Next Class

- Third-party web APIs and token-based authentication (8.1–8.2)
- Password hashing (8.3)